

\CSIS 2300 Introduction to Algorithms

Spring 2026 – Homework 5 – Greedy algorithms

Instructions: You do not need to typeset your solutions; however, your submission must be a single PDF file. The assignment is due by **8:59pm on Tuesday, February 24**. Remember that you may submit as many times as you wish before the deadline (we'll only grade the last one), but late submissions will receive no credit – so submit early, submit often!

This assignment consists of problems numbered 1-6; be sure to look at the second page.

(1) *(Counts double because it has four parts)* Let G be a connected, undirected, weighted graph. Prove or disprove each of the following statements. (You may assume the correctness of Prim's and Kruskal's algorithms if needed.)

- (a) If the weights of G are all distinct, then G has precisely one minimum spanning tree.
- (b) If the weights of G are not all distinct, then G must have more than one MST.
- (c) If e is a (not necessarily unique) minimum-weight edge in G , then it must be contained in at least one MST of G .
- (d) If e is the uniquely lowest-weight edge in G , then it must be contained in every MST.

(a) is true. Assume for contradiction that it is not, and there are two distinct minimum spanning trees T and T' . There must then be some edge e that is in T but not T' . If we add e to T' , this must create a cycle – call it C . Because edge weights are unique, there are only two possible cases: (1) e is larger than some edge $e' \in C$. Then $T - \{e\} \cup \{e'\}$ is a spanning tree with less total weight than T . (2) e is smaller than every edge in C . Then we can obtain a spanning tree smaller than T' by replacing any edge in C with e . Either way, we have contradicted the minimality of T and T' . ■

(b) is false. An easy counterexample is the case when G itself is a tree – then there is only one spanning tree, and thus only one MST. More generally, we can add as many high-weight edges as we like to an arbitrary graph without changing the MST.

(c) is true. The short proof simply notes that Kruskal's algorithm might pick e as its first choice, and Kruskal always produces an MST. More formally, let T be any MST without e . Adding e to T creates a cycle. Since e is minimal, every edge in the cycle is at least as much weight as e ; removing any other edge e' of the cycle thus leaves a spanning tree whose weight is no higher than T and thus $T \cup \{e\} - \{e'\}$ is also an MST.

(d) is true, using one of the same arguments as (c).

(2) An individual owns and operates a small picture-framing business as the only employee. They have a set of n jobs, and they know how long each job will take (call these times $t_1, t_2, \dots, t_n$), but they can work on only one job at a time. The goal is to sequence these jobs to minimize the total amount of waiting time experienced by each customer. For example, if they decide the first three jobs should be t_5, t_8, t_2 , then customer #5 waits for a time equal to $w_5 = t_5$, customer #8 waits for time $w_8 = t_5 + t_8$, and customer #2 waits for time $w_2 = t_5 + t_8 + t_2$; we want to minimize $\sum_{i=1}^n w_i$.

Design an algorithm to sequence the tasks that minimizes this sum. Prove that your algorithm produces an optimal answer.

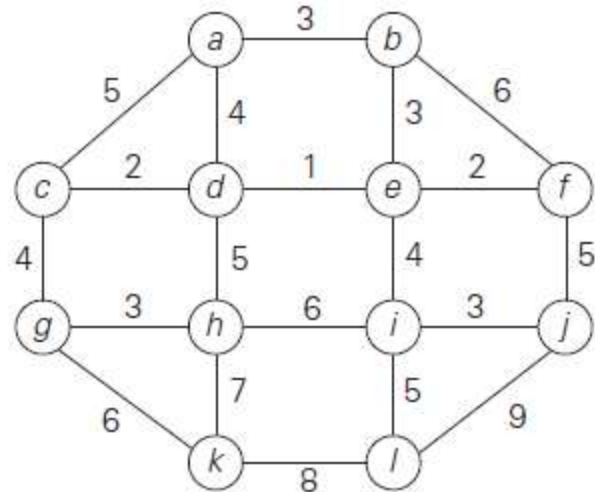
Sort the jobs in ascending order, so that $t_1 \leq t_2 \leq \dots \leq t_n$, and complete them in this order. The sum of all customers' wait times is then $nt_1 + (n-1)t_2 + (n-2)t_3 + \dots + 1t_n$ – the first job adds its completion time to the wait for every other job, the second adds its time to every job other than the first one, etc. We claim this sum is optimal (ties can be broken in any order without changing the sum).

Assume for contradiction that some other order is optimal. This order must have a pair of jobs with times $t_i < t_j$ with job j occurring before job i . (It's not $\leq$ because if they're equal, our algorithm might put them in either order.) Let $p_j < p_i$ be the positions of these jobs in this ordering. These jobs contribute $(n - p_j + 1)t_j + (n - p_i + 1)t_i$ to the overall waiting time. Observe that $(n - p_j + 1) > (n - p_i + 1)$; we are multiplying the larger time t_j by the larger factor. Thus, we can obtain a smaller overall sum by simply switching the positions of these two jobs – this will not affect the contributions from any other jobs in the list. But this contradicts the assumption that the new order with $p_j < p_i$ was optimal. Thus, every optimal ordering must have $p_i < p_j$ whenever $t_i < t_j$. ■

(3) Design an algorithm for finding the maximum spanning tree of an undirected weighted graph. Prove that your algorithm produces an optimal answer.

Modifying either of Prim or Kruskal to always select the largest available edge instead of the smallest works fine here. The proof of optimality should parallel the ones we discussed in class.

(4) Demonstrate the operation of Dijkstra's algorithm on the graph shown at right, starting at node a . Be sure to show each step of the process as well as giving the final result.



a (0, n/a)
 b (3, a)
 d (4, a)
 c (5, a)
 e ~~(6, b)~~ (5, d)
 f ~~(9, b)~~ (7, e)
 h (9, d)
 g (9, c)
 i (9, e)
 j (12, f)
 k ~~(16, h)~~ (15, g)
 l (14, i)

(5) Prove or disprove each of the following statements about Huffman codes:

- (a) The two least frequent symbols always have codewords that are the same length.
- (b) A more-frequent symbol cannot have a longer codeword than a less-frequent symbol.

(a) This is true if the frequencies are unique. With three or more symbols that all have the same lowest frequency, this might not be true. As a simple counterexample, consider an alphabet with three symbols that each occur $1/3$ of the time.

(b) *Do not score this problem – students were told to skip it.*

(6) A deck of 45 cards consists of one 1, two 2s, three 3s, and so on up to nine 9s. The deck is shuffled and a card is drawn at random. You are allowed to ask yes/no questions in an attempt to identify the card.

Design a strategy that minimizes the average (i.e. expected) number of questions you need to ask before identifying the card.

The problem is solved by building a Huffman tree; the correct tree is shown below. The values in the circles are the number of possible cards remaining after each question is asked; the squares are final answers.

The first question would then be: “Is your card a 4, 5, or 9?” If that answer is yes, the next question would be “Is your card a 9?”; if the initial answer is no, the next question would be “Is your card a 7 or 8?”

The mean number of questions needed to uniquely identify the card is then:

$$\frac{1}{45} (2 \cdot 9 + 3 \cdot (4 + 5 + 6 + 7 + 8) + 4 \cdot 3 + 5 \cdot (1 + 2)) = \frac{135}{45} = 3$$

